R Style Guide

Ivan Jacob Agaloos Pesigan

Follow the Google R style guide (https://google.github.io/styleguide/Rguide.html) as much as possible. The Google R style guide is a fork of the Tidyverse style guide (https://style.tidyverse.org/index.html), most notably with regard to function names. This guide lists the rules that differ from both or that they leave open. Where this guide and the Google guide disagree, this guide wins (for example, the Google guide asks for an explicit `return()`; this guide does not allow it).

`make style` formats the code with `styler` (Tidyverse style). `make lint` checks it with `lintr`'s default linters and the checks of this guide (Section 5). Agents follow `.setup/ai/style/r.md`, which follows this guide and is changed together with it.

1 R Objects

- `snake_case` for object names except for function names

2 R Functions

2.1 Names

- use `BigCamelCase` for function names to distinguish functions from other objects
- append a dot (`.`) at the beginning for unexported functions (`.Foo`)
- append dot Check (`.Check`) at the beginning for unexported functions used for sanity checks (`.CheckFoo`)

- S3 methods are named `generic.class`, with lowercase class names (`print.betadelta`, `summary.betadelta`)

2.2 Calling Functions

- when calling functions from another package use the `::` notation (`stats::rnorm(n = 10, mean = 100, sd = 15)`), except for functions from the `base` package. This includes the packages R attaches by default (`stats::`, `utils::`, `methods::`, `graphics::`, `grDevices::`) and `testthat::` in tests. Roxygen examples may call functions without `::`, since users copy them.

- when calling a function, name every argument, including the first (`length(x = x)`, `stats::lm(formula = y ~ x, data = df)`). Values passed through `...` are not named, since a name there changes the result (`c(1, 2)`, `list(a, b)`, `paste(a, b)`). The description and code of `testthat::test_that()` are the one exception (the skeleton in Section 3.2).

- do not use `library()`, `require()`, or `attach()` in `R` or in tests; use `pkg::fn()`

- do not use pipes (`|>`, `%>%`) in `R`. Scripts and vignettes may use them.

```
# yes
fit <- stats::lm(
  formula = QUALITY ~ NARTIC + PCTGRT,
  data = nas1982
)
out <- BetaDelta(object = fit, type = "mvn")
testthat::expect_equal(
  object = length(x = stats::coef(object = out)),
  expected = 2
)


# no
```

```
fit <- lm(QUALITY ~ NARTIC + PCTGRT, data = nas1982)
out <- BetaDelta(fit, type = "mvn")
testthat::expect_equal(length(coef(out)), 2)
```

2.3 Function Bodies

- do not use an explicit `return()`. The value of a function is its last expression.
- to exit early, put the rest of the body in the `else` branch, so that the value of the `if`–`else` is the result. Invalid input stops in a sanity check (`.CheckFoo()`) with `stop()`. In loops, `next` and `break` are fine. In tests, the `testthat::skip_*()` functions exit early (Section 3.2.4).
- every `if`, `else`, `for`, `while`, `repeat`, and `function` body is in braces ({}), even a body of one line
- keep lines under 80 characters, including roxygen comments

```
# yes
.Foo <- function(x) {
  if (length(x = x) == 0) {
    NULL
  } else {
    sum(x)
  }
}


# no
.Foo <- function(x) {
  if (length(x) == 0) return(NULL)
  sum(x)
}
```

2.4 Documentation

- every function has complete roxygen documentation, so that it can be understood from its file: a title, a description, `@author`, every `@param`, `@return`, and `@examples`
- exported functions also have `@export` and a test file (Section 3.2)
- unexported functions (`.Foo`, `.CheckFoo`) also have `@noRd`: documented in the source, with no help page
- `@author` names who wrote the code
- `@noRd` examples are not run, so keep them true when the function changes

The check for unexported functions is not yet in `make lint`. The order is rProject, the helper libraries, then each other project at its cleanup push.

2.5 Helper Libraries

A helper library (for example `linearAlgebra`) follows this guide as any package does.

- every function has a test file, exported or not
- tests that pass on the old code come before restructuring a body (removing `return()`, for example)
- a copy in a project's `.setup/r-dependencies/` is changed only in its library; a project never edits or restyles a copy

`make style` does not yet skip copies.

3 File and Filenames

- filenames should be in lowercase using dash (`-`) as separator
- capitalize `R` if used as a file extension

3.1 Files that Contain Functions

3.1.1 Folders

- `.setup/latex/r-scripts` for functions used to process LaTeX output
- `R` and `.setup/r-dependencies` for functions that are part of the package
- `.setup/r-miscellaneous` for useful functions but are not essential to the package

3.1.2 Files and Filenames

- use one file per function
- use the function name as part of the filename, converting `BigCamelCase` to lowercase using dash (`-`) as separator
- for files under `R` and `.setup/r-dependencies`
  - package name dash function name dot R (`template-foo.R`)
  - if the file contains an unexported function, follow the rules as usual but append the word `dot` at the end of the filename (`template-foo-dot.R`)
  - if the file contains an unexported sanity check function, follow the rules as usual but append the word `check` at the end of the filename (`template-foo-check.R`)
  - the S3 methods of one class go together in one file, an exception to one file per function (`template-foo-methods.R`)

3.2 Files Under Test Folders

3.2.1 Folders

- `tests/testthat` for unit tests that are part of the package
- `.setup/tests-benchmark` for benchmark tests that are **NOT** part of the package unit tests

- `.setup/tests-external` for external tests that are **NOT** part of the package unit tests, including tests that take too long for continuous integration (CI)

3.2.2 Files and Filenames

- use `tests/testthat/helper-template.R` (package name after `helper-`) for helper functions and objects used by the tests, such as the skip helpers (Section 3.2.4). `testthat` sources the `helper*.R` files before the `test*.R` files.
- use one file for tests for a single function
- use the function name as part of the filename but convert `BigCamelCase` to lowercase using dash (`-`) as separator
- every file follows the patterns below; there are no exceptions (for example, a coverage test is `test-template-coverage.R`, not `test-zzz-coverage.R`)
- for files under `tests/testthat`
  - test dash package name dash function name dot R (`test-template-foo.R`)
- for files under `.setup/tests-benchmark`
  - test dash benchmark dash package name dash function name dot R (`test-benchmark-template-foo.R`)
- for files under `.setup/tests-external`
  - test dash external dash package name dash function name dot R (`test-external-template-foo.R`)

3.2.3 Test Files

Each test file wraps its tests in `lapply()`, with the filename (without `.R`) as `text`. The functions are called as in the package code: `testthat::` and named arguments.

```
## ---- test-template-foo
lapply(
```

```
  X = 1,
  FUN = function(i,
                 text) {
    testthat::skip_on_cran()
    testthat::test_that(
      desc = paste(text, "adds one"),
      code = {
        testthat::expect_equal(
          object = Foo(x = 1),
          expected = 2
        )
      }
    )
  },
  text = "test-template-foo"
)
```

3.2.4 Skipping Tests

Run every test on CI (GitHub Actions) as far as possible. Skip a test on a capability (a program, a package, the network), not on a place. A skip is one line: at the top of the `lapply()` function when all of the tests in the file share it, or at the top of a `testthat::test_that()` otherwise.

Kind	Local	CI	CRAN	Skip
ordinary	run	run	skip	`testthat::skip_on_cran()`
smoke	run	run	run	none (`test-template-smoke.R`)
slow	skip	run	skip	`.SkipSlow()`
proprietary software	if found	never	skip	`.SkipIfMissing(program = "mplus")`
too long for CI	on request	never	never	`.setup/tests-external`

- **CRAN.** Tests are skipped on CRAN, except one fast smoke test file per package (`test-template-smoke.R`) that checks the main functions run.
- **Slow tests** run on CI and are skipped locally. `.SkipSlow()` runs a test when `CI` or `RPROJECT_SLOW_TESTS` is `"true"`; run them locally with `RPROJECT_SLOW_TESTS=true make all`.
- **Proprietary software** (Mplus, for example) never runs on CI. `.SkipIfMissing()` skips on CI, and elsewhere when the program is not found. These tests run locally or on Roar.
- **Tests too long for CI** are not unit tests: they go in `.setup/tests-external` and run on request (locally or on Roar).
- other skips use `testthat`: `skip_if_not_installed()` (a suggested package), `skip_if_offline()` (the network), `skip_on_os()` (the platform), `skip_if()` (a credential in an environment variable, never in the test)
- the helpers live in `tests/testthat/helper-template.R`, since a CRAN package cannot depend on `rProject`

4 Rcpp Functions

- Save `Rcpp` functions in `.setup/cpp` using filename conventions used for `R` except for the `.cpp` extension. Save `Makevars` in `.setup/cpp` as well.
- Use forward declarations of functions in `.setup/cpp/000-forward-declarations.cpp`.
- `Rcpp` files are styled using `clang-format` with the `style=google` flag. Make sure to limit the number of characters to less than 80 for each line especially when using `Roxygen` comments to avoid styling errors.
- `Rcpp` functions are unit tested as `R` functions.

5 Tools

- `make style`: `styler` (Tidyverse style) for R; `clang-format` (Google style) for C++. `styler` formats code; it does not add `::`, argument names, or braces.
- `make lint`: `lintr`'s default linters, with `BigCamelCase` and `snake_case` names, and the checks of this guide: no `return()`, `::`, named arguments, braces, no pipes in `R`, no `library()`, `require()`, or `attach()`, and filenames.
- `# nolint` only with the rule and a reason on the same line, for example `# nolint: object_name_linter. Reason: name fixed by the API.`; otherwise fix the code

References

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. https://www.R-project.org/